Reproducibility: execution record and recovery checks

Reproducibility: execution record and recovery checks I

This section is a machine-verifiable reproducibility certificate. Every value below is computed by the analysis pipeline and injected at render time, establishing a chain of custody from configuration through code to publication. The discipline is the one that gates this project's CI: every prose number is a generated token, every token is emitted by one generator function, and any drift between narrative and computed result fails the build before a green PDF exists (Peng 2011).

Determinism contract for seeded scientific results I

Reproducing every reported number requires only two recorded inputs: the global seed pinned here and the software environment fingerprinted in the next subsection. The determinism contract fixes the first — it states exactly what is held constant, what is asserted to machine tolerance, and what is deliberately not claimed byte-identical.

Determinism contract for seeded scientific results I

- Global seed: 0, threaded through every `np.random.default_rng(seed)`; the global `np.random` state is never used.
- Recovery identities use exact or machine-tolerance assertions; seeded study reports are regression-tested for repeatability under the recorded software environment. Rendered PDF/HTML/slide containers are validated as fresh publication products but are not claimed byte-identical across toolchain versions.
- No mocks anywhere: every test is a genuine computation on small categorical distributions or a seeded simulation under this repository's explicit no-mocks policy.

Environment fingerprint for the reported run I

The second reproduction input is the exact toolchain. Every field below is captured by the successful full test-and-coverage receipt before final variable generation rather than transcribed by hand. The receipt is bound to the source, tests, manuscript, source-owned documentation, release metadata, ISC tree, dependency lock, and fresh analysis receipt. It rejects any pre/post-suite drift in that boundary, so a reader matching this environment and the seed above can reproduce the seeded results; the config hash lets them confirm they are running the configuration from which this manuscript was rendered.

Environment fingerprint for the reported run I

Table 1: Software and configuration fingerprint for the hydrated manuscript. The build epoch is derived from `SOURCE_DATE_EPOCH`; an unreleased build records an explicit omitted sentinel rather than wall-clock time.

Field	Value
Python	3.13.11
NumPy	2.4.2
SciPy	1.18.0
PyTorch (MLP complement)	2.12.1
Platform	Darwin arm64
Config hash (SHA-256, first 16)	72ab9bf43b9f7914
Reproducible build epoch (UTC)	omitted (unreleased reproducible build)

Environment fingerprint for the reported run I

The exact environment used for the reported run is recorded in tbl. 1.

Reader-surface accessibility boundary I

The validated HTML manuscript is the canonical accessibility-enhanced reading surface. Its source gate requires a page language and title, a skip link and main landmark, non-empty image alternatives, figure captions, labelled full-size links, unique identifiers, resolved references, and present local assets on every generated page. These deterministic checks are not a claim of WCAG conformance: alternative-text quality, contrast, keyboard behavior, reading order, reflow, mathematics, and assistive-technology behavior still require manual review.

Reader-surface accessibility boundary I

The combined manuscript PDF is generated through the source-controlled LuaLaTeX/tagpdf path and is released only when `pdfinfo` reports `Tagged: yes`, qpdf exposes a non-empty `/Lang` and `StructTreeRoot`, and the source-bound language check passes. Some Poppler builds omit the language line from `pdfinfo` even when `/Lang` is present. The separate slide PDFs are checked structurally, textually, through retained renderer logs, and by raster inspection, but do not inherit the manuscript tagging status. Tagged structure is not PDF/UA conformance: a PDF/UA claim requires a dedicated conformance report plus screen-reader and reading-order review; `qpdf` structure checks and successful text extraction alone are insufficient.

Test and coverage evidence for the claim surface I

Test and coverage evidence for the claim surface I

- Acceptance criteria: 259 total, 257 passing.
- Project test suite: 1695 collected cases; the bound successful receipt records zero failed cases. The project no-mocks policy remains a separately executable source contract.
- Line coverage on `src/`: 90.26% (achieved by the bound full gate; $\geq 90\%$ line coverage is enforced in CI, while branch coverage is tracked separately in CI).

Test and coverage evidence for the claim surface I

To regenerate this evidence from a clean checkout, run the project suite under the pinned development environment; the same invocation is the CI gate, so a passing local run and a green build are the same event:

```
uv run --extra dev pytest tests/ \
  --cov=src --cov-fail-under=90
```

Test and coverage evidence for the claim surface I

For a release-facing hydration, use the receipt-producing wrapper after any required provisional pre-test render, then rerun hydration without its provisional flag:

```
uv run --extra dev python scripts/validate_test_coverage.py
uv run python scripts/z_generate_manuscript_variables.py
```

Artifact inventory for figures, data, and reports I

Artifact inventory for figures, data, and reports I

Table 2: Top-level generated files in `output/figures`, `output/data`, and `output/reports` at token-hydration time. The generated release manifest is the source of truth for the larger recursive publication bundle. Artifacts are regenerable reviewer snapshots and must not be hand-edited.

Category	Count
Figures	61
Data files	6
Reports	32
Total	99

Artifact inventory for figures, data, and reports I

The top-level artifact counts in tbl. 2 complement the recursive, checksum-bearing release manifest.

Recovery-limit certificate for the client and project-pool corners I

The recovery identities are reproducibility checks: the client machinery must return to standard Bayes at its KL/NLL loss limits, and the server heuristic must return to the project's log-linear pool at zero robustness (eq. 7, eq. 6). Under the explicit shared-support, posterior-log-potential, and fixed-weight assumptions of sec. 5.8, that pool specializes Eq. 7's message-combination term; it does not reproduce the complete source protocol. These deviations are computed on every build:

Recovery-limit certificate for the client and project-pool corners I

- `robust_aggregate(robustness=0)` versus `log_linear_pool` (eq. 6): 0
- `generalized_posterior(KLD, NLL)` versus closed-form Bayes: 5.55e-17
- Rényi divergence versus KL as $\alpha \to 1$: 0
- β-loss versus NLL as $\beta \to 0$: 0
- rcce versus NLL as $q_{\mathrm{loss}} \to 0$: 0

Recovery-limit certificate for the client and project-pool corners I

Any drift in these limits beyond machine precision would mean the robust generalization no longer contains its standard-Bayes client limit and project-local log-linear-pool server limit (sec. 7.1) — and would fail the core test suite before this certificate could render. The certificate covers the recovery identity and the client-side result under the cited source theorem's matching assumptions only; the server-side `robust_aggregate` heuristic is certified here for its recovery limit alone, not for any bounded-influence property (sec. 8.13).

Recovery-limit certificate for the client and project-pool corners I

All code is authored by Daniel Ari Friedman and licensed under the MIT license. This is project version 1.0.4.